RLevator: Using Reinforcement Learning to Make a Smarter Elevator

Tejas Narayanan* (tejasn9), Kiran Bhat* (kvbhat)

December 10, 2022

1 Introduction

Current elevator algorithms often make decisions that force us to wait longer than otherwise necessary. Can we do better? In this project, we aim to use deep reinforcement learning to train an agent to make more dynamic elevator control decisions that minimize passenger waiting times. We define passenger waiting time as the time between a passenger calling the elevator and arriving at their target floor.

The input to our algorithm is an observation array containing only information that traditional elevators have access to, including the elevator's current floor, its direction of motion, and the buttons currently pressed inside and outside of the elevator. We then use a neural network to determine which floor the elevator should target. The neural network's weights are updated using policy gradient methods.

The code for RLevator is available at `https://github.com/tnarayanan/RLevator`. Instructions for running the code can be found in the `README.md` file

2 Background

In reinforcement learning (RL), an *agent* takes *actions* in an *environment* and receives a *reward* for its action in that state. The agent's goal is to maximize its expected discounted reward over the environment's lifespan, or an *episode*:

$$\mathbb{E}[R(s_0) + \gamma R(s_1) + \gamma^2 R(s_2) + \cdots + \gamma^T R(s_T)] \qquad \text{where } 0 \leq \gamma < 1$$

Formally, the agent receives an observation of the environment, or *state*. It then chooses its action a to be the output of a *policy* function $\pi(s)$, which maps states to actions. There are many approaches for finding a good policy function, such as policy iteration as well as policy gradient methods like Proximal Policy Optimization (PPO) [1]. Next, the environment applies action a and returns both the next state s' and the reward for taking action a in the current state s, or $R(s, a)$. This process repeats until the environment ends the episode, often after a failure case or a pre-defined number of timesteps.

3 Related Work

Prior work has been conducted in the area of reinforcement learning for elevator control. Each work has notable differences compared to our methods. Crites and Barto [2] chose to minimize only the time between a passenger making a request and entering the elevator, whereas our method minimizes the total time between making a request and exiting the elevator at the desired floor. Li [3] specifically simulated a down-peak traffic pattern, in which requests were randomly made at each floor but were all going to the ground floor. On the other hand, our environment generates random requests between all floors, making it more generalizable to all request patterns. Yuan et. al [4] utilized Q-learning compared to our policy gradient methods, and also simulated a down-peak pattern. Zheng et. al [5] similarly used Q-learning, but considered additional factors such as energy consumption when crafting the reward function.

We also researched existing fixed elevator control algorithms in order to implement one or more as a baseline against which we could compare our RL agent's performance. One such algorithm is Karp's elevator algorithm [6], which is described in further detail in section 4.2.

4 Methods

4.1 Environment

We created an environment using Gym [7], a framework created by OpenAI to build and test reinforcement learning environments and policies.

The environment simulates a building with n floors. The elevator can be controlled through a *target floor*, which is the floor that it will move to and open at next. The elevator takes one timestep to travel between two adjacent floors.

A new elevator request between two floors is randomly generated each timestep with a probability of 0.3.

4.1.1 Observation Space

An agent acting in the environment receives the following information as an observation:

- the floor the elevator is currently on
- the floor the elevator is currently targeting
- the direction the elevator was previously traveling in
- the direction the elevator is traveling
- for each floor k:
  - whether or not the button for floor k is pressed in the elevator
  - whether or not the up button is pressed on floor k
  - whether or not the down button is pressed on floor k

4.1.2 Action space

The action space is a single number between 0 and $n - 1$, representing the target floor for the elevator.

4.1.3 Reward function

Let r_w be the reward applied per waiting passenger per timestep. Our reward function is as follows:

$$r_t = (r_w \times \#\text{ of waiting/riding passengers at timestep } t)$$

In other words, each passenger contributes r_w reward per timestep they are waiting. In our experiments, we set $r_w = -1$.

4.2 Non-RL Baseline

We implemented Karp's algorithm [6] (often known simply as the "elevator algorithm"), which is commonly used in real-world elevator systems (such as the Escondido Valley Graduate Residences at Stanford), as a fixed, non-RL baseline. In this algorithm, an elevator will continue to move in its current direction until it has fulfilled all requests in that direction. Then the elevator will either switch directions, or become idle if there are no requests anywhere in the building. This serves as a competitive algorithm to compare the RL agent's performance against.

4.3 RL agent

We set up a policy to learn an elevator control algorithm dynamically. The agent acts in the previously-described environment.

4.3.1 Policy

We use the Stable Baselines 3 library [8], which has built-in RL training algorithms and policies that interface with OpenAI Gym. The policy and value networks are three-layer multi-layer perceptrons, with input and output dimensions matching the size of the observation and action spaces respectively as well as a hidden layer with 64 neurons.

4.3.2 Policy updates

By using Stable Baselines 3, we are able to select from a variety of policy update algorithms including PPO [1], which we chose for this agent.

Traditional policy gradient methods such as trust region policy optimization (TRPO) [9] update the policy to increase expected reward subject to a constraint of how large the update can be. However, these methods are extremely sensitive to their update size and learning rates and are also very expensive, so they are often difficult to use in practice. To fix these issues, PPO updates the policy to increase expected reward as before, but clips the policy update based on a hyperparameter rather than constraining the updates. This is much cheaper to compute, and also leads to empirically better results.

4.3.3 Curriculum Learning

In complex reinforcement learning environments, it is often challenging for an agent to learn a successful policy. Curriculum learning [10] is a method of gradually increasing the complexity of an environment up to its final form, and only increasing the complexity once the agent has learned a successful policy for the current version of the environment. We utilize curriculum learning for one of our experiments in which the environment is more complex.

4.4 Evaluation

We evaluate the performance of each control algorithm by running the control algorithm for 100 episodes, then computing the mean episodic reward and the standard deviation of the episodic reward across all 100 episodes. The mean episodic reward is a measure of how time-efficient the control algorithm was on average. If the mean episodic reward is higher (closer to 0), the average wait time of passengers is lower. The standard deviation of the episodic reward describes how consistently the control algorithm performed across the varying request patterns of each episode.

For this environment, we chose the episode length to be 100 timesteps.

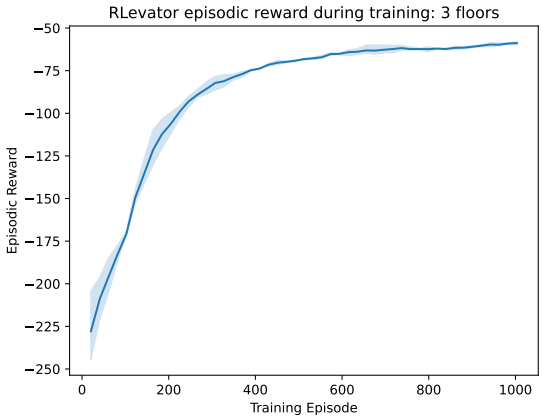
5 Experiments & Results

5.1 Agent performance on simple environments

For initial experimentation, we trained an RL agent on two simple versions of the environment:

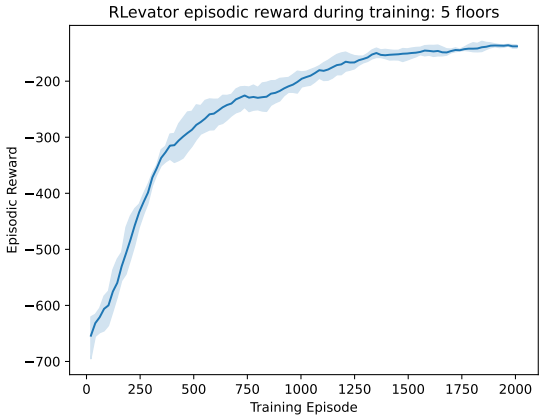
- 3 floors, trained for 1000 episodes
- 5 floors, trained for 2000 episodes

We trained the agent on each environment three times, with different random seeds, and averaged the results. We compared each agent's performance to Karp's algorithm's performance on the same environment.



Control Algorithm	Mean Episodic Reward	Std. Dev
Karp	-195.54	70.03
RLevator (three seed average)	-58.24	11.78

Figure 1: Training and evaluation performance, 3 floors



Control Algorithm	Mean Episodic Reward	Std. Dev
Karp	-322.08	93.50
RLevator (three seed average)	-126.48	28.84

Figure 2: Training and evaluation performance, 5 floors

We see that in both the 3-floor and 5-floor environments, the episodic reward increased as training progressed. This indicates that the RL agent progressively updated its policy and thereby reduced passenger wait times. It appears that the training process was stable, since the reward increase over time was very consistent across the 3 random seeds (as seen in the training graphs in figures 1 and 2).

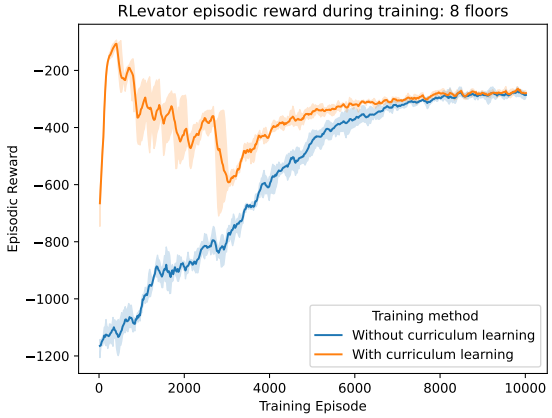
We note that in both environments, RLevator has a higher mean episodic reward and lower standard deviation in episodic reward than Karp's algorithm when evaluating the two control algorithms. This implies that RLevator not only reduces passenger wait times compared to Karp's algorithm, but also performs more consistently in response to varying request patterns.

5.2 Curriculum learning on more complex environments

After confirming that RLevator was able to outperform Karp's algorithm on simple environments, we decided to increase the complexity of the environment and test its performance. We set the environment to have 8 floors, and be trained over 10,000 episodes.

Given the increased complexity of the environment, we also tested the effects of applying curriculum learning [10]. We initialized the environment with 3 floors. Once the agent was able to surpass a reward threshold t_f when acting in an environment with f floors, we increased the number of floors by 1. Once the environment reached 8 floors, we stopped updating the environment via the curriculum. We defined $t_f = -85 - 50(f - 3)$; this function was experimentally determined.

Note that for higher f, the agent does not require as much reward in order to progress in its curriculum. This is because taller buildings inherently increase the total amount of waiting experienced by passengers, so the reward ceiling decreases as the number of floors increases.



Control Algorithm	Mean Episodic Reward	Std. Dev
Karp	-480.97	109.10
RLevator (three seed average)	-265.85	60.40
RLevator w/ curriculum (three seed average)	-266.06	62.60

Figure 3: Training and evaluation performance, 8 floors

Both the RLevator algorithms had a much higher mean episodic reward than Karp's algorithm, indicating that they were still able to learn successful policies on the more complex version of the environment.

The initial decreases in reward for the curriculum learning agent were due to performance struggles each time the curriculum was updated, since the agent was progressively discovering more features of the environment (more floors). However, it quickly recovered after each curriculum update in order to progress to the next level of the curriculum. Moreover, we notice that both of the RLevator algorithms (with and without curriculum learning) converged to very similar episodic rewards, but RLevator with curriculum learning converged in fewer training episodes than RLevator without curriculum learning. This shows that curriculum learning made the training process easier for the agent in the context of a complex environment.

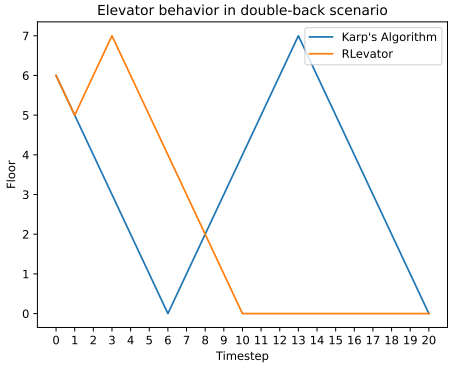
5.3 Behavior in double-back scenario

Let us consider the following scenario (which we will refer to as the double-back scenario):

- There is a passenger who just entered the elevator on the 6th floor and requested to go to the ground floor (0th floor). The elevator is about to move down.
- At the same time, a group of passengers on the 7th floor request to go down.

In this case, Karp's algorithm would recognize that the elevator is moving down, and therefore continue to move downwards until dropping off the passenger at the ground floor. Meanwhile, the group of passengers on the 7th floor is still waiting. After dropping off the passenger, the elevator goes back up to the 7th floor, and then brings the group to the ground floor. However, since our performance objective is to minimize the total wait time of all passengers, the elevator should have initially moved from the 6th to the 7th floor to pick up the group, then moved down to the ground floor to drop everyone off.

On the other hand, we observed that the trained RLevator agent reversed the direction of the elevator and picked up the group of passengers waiting on the 7th floor before moving to the ground floor.



6 Conclusion and Future Work

We found that RLevator, a dynamically-learned elevator control algorithm, outperformed the currently-used fixed control algorithm in simple environments with a small number of floors as well as difficult environments with more floors. Moreover, applying curriculum learning techniques increased the convergence speed of the RL agent during training.

However, it is important to note that minimizing passenger wait time may have undesirable side effects, such as reducing passenger satisfaction. For example, RLevator's approach in the double-back scenario does indeed reduce average wait times for passengers, but it does so in a way that may feel unfair to certain passengers. As the elevator is moving down, the passenger inside may expect to travel all the way to their destination on the ground floor. However, they may be upset if the elevator unexpectedly goes back up to pick up extra passengers before going back to the ground floor. While the total passenger wait time is reduced, the satisfaction of the original passenger will likely be very low, especially if they were in a rush.

More generally, the success metric of an elevator control algorithm may not purely be numerical; other qualitative factors such as human perception of time and fairness in completing requests may also be important factors, depending on the scenario in which the algorithm is deployed.

There are numerous areas in which work can be done to extend RLevator's success. One such area is the multi-elevator domain. The RLevator codebase already has support for multi-elevator systems, so experiments can be conducted to quantify RLevator's performance on these systems compared to Karp's algorithm. Moreover, changes to the reward function can be made in order to consider human factors as discussed in section 5.3. While the effect of these changes may be more difficult to see numerically, they would be much more apparent when qualitatively evaluating the models.

Since the agent may occasionally behave erratically or unpredictably, it may not be practical to deploy the learned elevator control algorithm directly. However, analyzing the agent's decisions can help identify time-saving techniques to develop better fixed-control algorithms in the future.

7 Contributions

All members contributed equally to this project by conducting initial research, creating the environment, training agents, and writing the final paper.

References

[1] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, *Proximal policy optimization algorithms*, Number: arXiv:1707.06347, Aug. 28, 2017. DOI: 10.48550/arXiv.1707.06347. arXiv: 1707.06347[cs]. [Online]. Available: http://arxiv.org/abs/1707.06347.

[2] R. Crites and A. Barto, "Improving elevator performance using reinforcement learning," in *Advances in Neural Information Processing Systems*, vol. 8, MIT Press, 1995. [Online]. Available: https://proceedings.neurips.cc/paper/1995/hash/390e982518a50e280d8e2b535462ec1f-Abstract.html.

[3] H. Li, "The implementation of reinforcement learning algorithms on the elevator control system," in *2015 IEEE 20th Conference on Emerging Technologies & Factory Automation (ETFA)*, ISSN: 1946-0759, Sep. 2015, pp. 1–4. DOI: 10.1109/ETFA.2015.7301554.

[4] X. Yuan, L. Buşoniu, and R. Babuška, "Reinforcement learning for elevator control," *IFAC Proceedings Volumes*, 17th IFAC World Congress, vol. 41, no. 2, pp. 2212–2217, Jan. 1, 2008, ISSN: 1474-6670. DOI: 10.3182/20080706-5-KR-1001.00373. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S1474667016392783.

[5] L. zheng, S. Guang, and D. Hui, "Research of elevator group scheduling system based on reinforcement learning algorithm," in *Proceedings of 2013 2nd International Conference on Measurement, Information and Control*, vol. 01, Aug. 2013, pp. 606–610. DOI: 10.1109/MIC.2013.6758037.

[6] D. E. Knuth, *The Art of Computer Programming*, 2nd ed. Reading, Mass: Addison-Wesley, 1997, vol. 3, ISBN: 9780201896831.

[7] G. Brockman, V. Cheung, L. Pettersson, *et al.*, *OpenAI gym*, Number: arXiv:1606.01540, Jun. 5, 2016. DOI: 10.48550/arXiv.1606.01540. arXiv: 1606.01540[cs]. [Online]. Available: http://arxiv.org/abs/1606.01540.

[8] A. Raffin, A. Hill, A. Gleave, A. Kanervisto, M. Ernestus, and N. Dormann, "Stable-baselines3: Reliable reinforcement learning implementations," *Journal of Machine Learning Research*, vol. 22, no. 268, pp. 1–8, 2021, ISSN: 1533-7928. [Online]. Available: http://jmlr.org/papers/v22/20-1364.html.

[9] J. Schulman, S. Levine, P. Moritz, M. I. Jordan, and P. Abbeel, *Trust region policy optimization*, Apr. 20, 2017. DOI: 10.48550/arXiv.1502.05477. arXiv: 1502.05477[cs]. [Online]. Available: http://arxiv.org/abs/1502.05477.

[10] Y. Bengio, J. Louradour, R. Collobert, and J. Weston, "Curriculum learning," in *Proceedings of the 26th Annual International Conference on Machine Learning - ICML '09*, Montreal, Quebec, Canada: ACM Press, 2009, pp. 1–8, ISBN: 978-1-60558-516-1. DOI: 10.1145/1553374.1553380. [Online]. Available: http://portal.acm.org/citation.cfm?doid=1553374.1553380.